

Limited support for `constexpr void*`

Document #: P2747R0
Date: 2022-12-16
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Introduction](#)
- [2 Type Erasure](#)
- [3 Placement new](#)
- [4 Uninitialized objects](#)
- [5 Proposal](#)
 - [5.1 Implementation Concerns](#)
- [6 References](#)

§ 1 Introduction

One of the operations that you are not allowed to do during constant evaluation today is, from [\[expr.const\]/5.14](#):

(5.14) a conversion from type cv `void*` to a pointer-to-object type;

This makes some amount of sense from the perspective of wanting constant evaluation to be a safer subset of C++, we can't just go throwing away all of our type information. But not all conversions from `void*` are the same - some are, in fact, perfectly safe, and the lack of ability to do so prevents some useful tools from being available at compile time.

§ 2 Type Erasure

Consider the following [very] reduced implementation of `function_ref`:

```
template <typename R, typename... Args>
class function_ref<R(Args...)> {
    void* data;
    auto (*func)(void*, Args...) -> R;

public:
    template <typename F>
    constexpr function_ref(F&& f)
        : data(&f)
```

```

    , func(+[](void* f, Args... args){
        using FD = std::remove_reference_t<F>;
        return (*static_cast<FD*>(f))(args...);
    })
{ }

constexpr auto operator()(Args... args) const -> R {
    return func(data, args...);
}
};

```

This is a common technique for type erasure - we have a `void*` that type erases some data and then we have a function pointer that knows how to cast that `void*` to the appropriate type. `data` here may be a `void*`, but it *actually does* point to an `FD`. That cast right there is perfectly safe, by construction. But we're just not allowed to do it during constant evaluation time.

Which means that this entire implementation strategy just... doesn't work.

§ 3 Placement new

Consider this implementation of `std::uninitialized_copy`, partially adjusted from [cppreference](#):

```

template <input_iterator I, sentinel_for<I> S, nothrow_forward_iterator I2>
constexpr auto uninitialized_copy(I first, S last, I2 d_first) -> I2 {
    using T = iter_value_t<I2>;
    I2 current = d_first;
    try {
        for (; first != last; ++first, (void)++current) {
            ::new (std::addressof(*current)) T(*first);
        }
    } catch (...) {
        std::destroy(d_first, current);
        throw;
    }
}

```

This fails during constant evaluation today because placement new takes a `void*`. But it takes a `void*` that points to a `T` - we know that by construction. It's just that we happen to lose that information along the way.

Moreover, that's not *actually* how `uninitialized_copy` is specified, we actually do this:

```

- ::new (std::addressof(*current)) T(*first);
+ ::new (voidify(*current)) T(*first);

```

where:

```

template<class T>
constexpr void* voidify(T& obj) noexcept {
    return const_cast<void*>(static_cast<const volatile void*>
        (std::addressof(obj)));
}

```

Which exists to avoid users having written a global placement new that takes a T^* .

The workaround, introduced by [P0784R7], is a new library function:

```
template<class T, class... Args>
constexpr T* construct_at( T* p, Args&&... args );
```

This is a magic library function that is specified to do the same *voidify* dance, but which the language simply recognizes as an allowed thing to do. `std::construct_at` is explicitly allowed in [expr.const]/6:

- ⁶ [...] Similarly, the evaluation of a call to `std::construct_at` or `std::ranges::construct_at` ([specialized.construct]) does not disqualify *E* from being a core constant expression unless the first argument, of type T^* , does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of *E*, or the evaluation of the underlying constructor call disqualifies *E* from being a core constant expression.

It's good that we actually have a solution - we can make `uninitialized_copy` usable during constant evaluation simply by using `std::construct_at`. There's even a paper to do so ([P2283R2]). But that paper also had hinted at a larger problem: `std::construct_at` is an *extremely* limited tool as compared to placement new.

Consider the different kinds of initialization we have in C++:

kind	placement new	<code>std::construct_at</code>
value initialization	<code>new (p) T(args...)</code>	<code>std::construct_at(p, args...)</code>
default initialization	<code>new (p) T</code>	Not currently possible. [P2283R1] proposed <code>std::default_construct_at</code>
list initialization	<code>new (p) T{a, b}</code>	Not currently possible, could be a new function?
designated initialization	<code>new (p) T{.a=a, .b=b}</code>	Not possible to even write a function

That's already not a great outlook for `std::construct_at`, but for use-cases like `uninitialized_copy`, we have to also consider the case of guaranteed copy elision:

```
auto get_object() -> T;

void construct_into(T* p) {
    // this definitely moves a T
    std::construct_at(p, get_object());

    // this definitely does not move a T
    ::new (p) T(get_object());

    // this now also definitely does not move a T, but it isn't practical
    // and you also have to deal with delightful edge cases - like what if
    // T is actually constructible from defer?
```

```

struct defer {
    constexpr operator T() const { return get_object(); }
};
std::construct_at(p, defer{});
}

```

Placement new is only unsafe because the language allows you to do practically anything - want to placement new a `std::string` into a `double*`? Sure, why not. But during constant evaluation we already have a way of limiting operations to those that make sense - we can require that the pointer we're constructing into actually is a `T*`. The fact that we have to go through a `void*` to get there doesn't make it unsafe.

§ 4 Uninitialized objects

Since C++20, we can use `std::vector<T>` during constant evaluation. But what about a type that doesn't actually need allocation, like `static_vector<T, capacity>`? That proposal ([P0843R4]) currently only supports `constexpr` if `T` is trivially copyable and default constructible, because of issues with storage. David Stone covered this in a [recent CppCon talk](#).

The issue there is you want to do something like... this:

```

template <typename T, size_t Capacity>
class static_vector {
    alignas(T) std::byte storage_[sizeof(T) * Capacity];
    size_t size_;

public:
    constexpr auto data() -> T* {
        return (T*)storage_;
    }

    void push_back(T const& rhs) {
        // check size vs capacity, etc, out of scope
        ::new (data() + size_) T(rhs);
        ++size_;
    }
};

```

But here the problem is on top of the `void*` that we have to deal with from placement-new, we have the issue that the originating storage isn't actually a `T` to begin with, it's just an array of `std::byte` (or `char` or `unsigned char`). It can't just be a `T[Capacity]` because we want to avoid having to default-construct all of our `Ts`.

We could try to change the storage to from the array of `std::byte` to something like:

```

union U {
    constexpr U() { }
    constexpr ~U() { }

    T data[N];
} storage_;

```

This helps in the sense that our placement-new now does originate with a τ , so we don't have that particular concern. But we now have the added issue that at no point did we begin the lifetime of this union alternative. That is, consider this hilariously complex [identity function](#):

```
1 | constexpr int id(int i) {
2 |     struct X {
3 |         int i;
4 |         constexpr X(int i) : i(i) { }
5 |     };
6 |
7 |     union U {
8 |         constexpr U() { }
9 |         constexpr ~U() { }
10 |         X data[1];
11 |     };
12 |
13 |     U storage;
14 |     X* x = std::construct_at(&storage.data[0], i);
15 |     return x->i;
16 | }
17 |
18 | static_assert(id(42) == 42);
```

On line 14 there, while `&storage.data[0]` is actually an `X*`, the data union alternative of storage isn't active yet. Nothing constructed it. Nevertheless, as you can see in the compiler explorer link, gcc and msvc both accept this code.

How can we make this work? There's a few directions, I think.

First, we can have the language recognize an uninitialized storage type. Like a `std::uninitialized<T>`. Which, for the static_vector case, would be `std::uninitialized<T[Capacity]>` This would be a type that's just already much easier to use than the `std::aligned_storage` that we deprecated ([P1413R3]) simply on the basis that you can't misspell it. libstdc++, for instance, has an `__aligned_membuf<T>` type with a pretty [nice API](#) that could be the basis for this (except that the `ptr()` function should really return `remove_extent_t<T>*` not just `T*`, to properly handle the array case). This seems like a good direction since it makes intent that much clearer and would be harder to misuse (since you can't forget the alignment):

With buffer array	With <code>std::uninitialized</code>
<pre> template <typename T, size_t Capacity> class static_vector { alignas(T) std::byte storage_[sizeof(T) * Capacity]; size_t size_; public: constexpr auto data() -> T* { return (T*)storage_; } }; </pre>	<pre> template <typename T, size_t Capacity> class static_vector { std::uninitialized<T, N> storage_; size_t size_; public: constexpr auto data() -> T* { return storage_.data(); } }; </pre>

Second, we could simply bless the union example from earlier. That is, it's already the case that a placement new (or, for now, `std::construct_at`) on a union alternative changes that to be the active alternative. But we could extend that to be true for if you are constructing a subobject of an alternative (or, at the very least, an array member). This approach isn't actually mutually exclusive with `std::uninitialized` - it just makes `std::uninitialized` something that can be implemented outside of the standard library. Depending on your perspective, that's either very good (less magic things in the language that can only be implemented by the compiler) or very bad (allows for multiple different implementations of `std::uninitialized`).

An alternative to making the union example work is to provide a mechanism to start the lifetime of a union alternative but performing no initialization. One idea discussed in context of JF Bastien's [\[P2723R0\]](#) was the ability to explicitly denote no initialization, not as an attribute but using some kind of syntax. Some ideas that I've seen thrown around were:

```

void test() {
    int a;                // zero-initialized, per the paper
    int b = void;         // uninitialized (explicitly)
    int c = uninitialized; // uninitialized (explicitly)
}

```

The advantage of the syntax over the attribute in this particular case is that the storage example could be implemented as:

```

template <typename T>
union storage_for {
    constexpr ~storage_for() requires std::is_trivially_destructible_v<T> =
        default;
    constexpr ~storage_for() { }

    T data = void; // or whatever syntax
};

```

We'd still need it to be a union, because we need to avoid destruction, but at least this gives us an active union member that we could then safely use.

§ 5 Proposal

This paper proposes extending constant evaluation support to cover a lot more interesting cases by striking the rule about conversion from `cv void*`. We still require that reading through a `τ*` is only valid if actually points to a `τ` (as in, the `void*` had to have been obtained by a `static_cast` from a `τ*`) - this doesn't open the door for various type punning shenanigans during constant evaluation time. These are all conversions that would only be allowed if they were actually valid.

Allowing converting from `cv void*` to `cv τ*`, if there is actually a `τ` there, should immediately allow placement-new to work, but this may require explicit permission for global placement new in the same place where we currently have explicit permission for `std::construct_at`.

Lastly, this paper proposes that placement new on an array alternative of a union implicitly starts the lifetime of the whole array alternative. This seems consistent with the implicit-lifetime-type rule that we have for arrays, and is the minimal change required to allow `static_vector` to work and to allow user implementations of `uninitialized<T>`. I'm not proposing `std::uninitialized<T>` specifically because I don't think it's strictly necessary, and the shape of it will largely end up depending on the discussion around JF's paper - which is otherwise completely unrelated to this paper.

Importantly, the changes proposed here allow code that people would already be writing today to simply work during compile time as well. Well, for uninitialized storage people probably use an aligned array of bytes rather than a union with an array of `τ`, but least this isn't too far off, and the main point is that the proposal isn't inventing a new way of writing compile-time-friendly code.

§ 5.1 Implementation Concerns

One of the raised concerns against allowing conversion from `cv void*` in the way proposed by this paper is the cost to certain implementations of tracking pointers - specifically in the cost of validating this conversion. However, while everyone would obviously prefer things to be as fast to compile as possible, we're not choosing here between a fast approach and a slow approach - the decision here is between a slow approach and *no approach at all*. The inability to convert from `cv void*` means we can't have a certain class of type erasure, we can't do several kinds of placement new and other kinds efficiently, and we can't have a `constexpr static_vector` without introducing more special cases for magic library names. I don't think that's a good trade-off.

§ 6 References

[P0784R7] Daveed Vandevoorde, Peter Dimov, Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More `constexpr` containers.
<https://wg21.link/p0784r7>

[P0843R4] Gonzalo Brito Gadeschi. 2020-01-13. `static_vector`.
<https://wg21.link/p0843r4>

[P1413R3] CJ Johnson. 2021-11-22. Deprecate std::aligned_storage and std::aligned_union.

<https://wg21.link/p1413r3>

[P2283R1] Michael Schellenberger Costa. 2021-04-19. constexpr for specialized memory algorithms.

<https://wg21.link/p2283r1>

[P2283R2] Michael Schellenberger Costa. 2021-11-26. constexpr for specialized memory algorithms.

<https://wg21.link/p2283r2>

[P2723R0] JF Bastien. 2022-11-16. Zero-initialize objects of automatic storage duration.

<https://wg21.link/p2723r0>